

XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

二维数组与数组模拟链表

2D Arrays, Matrix Simulation and Array-Based Linked Lists

作者：Wu Youfan
School of Computing and Data Science, Xiamen University Malaysia

2026 年 10 月 30 日

目录

1. 二维数组基础
2. 矩阵基本操作
3. 螺旋矩阵
4. 矩阵旋转
5. 经典矩阵应用
6. 数组模拟链表

目录

⑦ 总结

目录

1 二维数组基础

2 矩阵基本操作

3 螺旋矩阵

4 矩阵旋转

5 经典矩阵应用

6 数组模拟链表

目录

7 总结

什么是二维数组

一维数组：一排格子，每个格子放一个元素，用 `a[i]` 访问。

二维数组：一个矩形表格，每个格子放一个元素，用 `a[i][j]` 访问，其中 i 是行号，j 是列号。

三个要点

- 行下标和列下标都从 0 开始
- n 行 m 列时，合法范围是 $0 \le i < n$ 且 $0 \le j < m$
- 内存中按“行优先”存放：第 i 行紧跟第 $i-1$ 行

二维数组 = 行列交叉定位的表格

定义与初始化

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

定义

- 全局：`int a[105][105];`，未写值时默认全 0
- 局部：`int a[105][105];`，未初始化，值随机
- 建议把大数组放全局，省去手动清零

初始化

- 全局数组默认全 0，最省心
- 局部数组可用 `memset(a, 0, sizeof(a))` 手动清 0
- 实际题目多数是边读入边使用，不必手写初值

大数组放全局，默认全 0，不需要手动清

遍历与输入输出

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

读入模板

- 先读入行数 n 和列数 m
- 外层循环 i 从 0 到 $n-1$，内层循环 j 从 0 到 $m-1$
- 每次读入 `a[i][j]`

输出模板

- 外层 i，内层 j，输出 `a[i][j]` 后加空格
- 每行结束后加换行

外层走行，内层走列，行优先是标准写法

目录

1 二维数组基础

2 矩阵基本操作

3 螺旋矩阵

4 矩阵旋转

5 经典矩阵应用

6 数组模拟链表

目录

7 总结

例题：矩阵转置

题目：给定一个 n 行 m 列的矩阵，输出它的转置矩阵（m 行 n 列）。

示例

- 输入 2 行 3 列，内容为 $\begin{matrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{matrix}$
- 转置后 3 行 2 列，内容为 $\begin{matrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{matrix}$

矩阵转置：解法

核心思路：新矩阵的第 i 行第 j 列等于原矩阵的第 j 行第 i 列。

关键步骤

- 读入原矩阵 $a[n][m]$
- 双重循环 i 从 0 到 $n-1$，j 从 0 到 $m-1$
- 令 `b[j][i] = a[i][j]`
- 输出新矩阵 $b[m][n]$

关键点：转置后行列维度互换，新矩阵是 m 行 n 列，输出时外层走 m、内层走 n。

行变列、列变行，下标对调即可

矩阵转置：代码

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```
const int MAXN = 105;
int a[MAXN][MAXN];
int b[MAXN][MAXN];

int main() {
    int n, m;
    scanf("%d %d", &n, &m);
    // 读入原矩阵
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            scanf("%d", &a[i][j]);
    // 转置：行变列、列变行
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            b[j][i] = a[i][j];
    // 输出转置矩阵，注意维度换成了 m 行 n 列
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++)
```

矩阵转置：代码

```
19              printf("%d ", b[i][j]);
20          printf("\n");
21      }
22      return 0;
23  }
```

目录

1 二维数组基础

2 矩阵基本操作

3 螺旋矩阵

4 矩阵旋转

5 经典矩阵应用

6 数组模拟链表

目录

7 总结

例题：螺旋矩阵

题目：给定 n 和 m，按顺时针螺旋顺序，从 1 开始依次填入 n 行 m 列的矩阵，输出最终矩阵。

示例 $(n = 3,\ m = 3)$

$$\begin{matrix} 1 & 2 & 3 \\ 8 & 9 & 4 \\ 7 & 6 & 5 \end{matrix}$$

螺旋矩阵：解法

核心思路：维护上下左右四个边界（初值 $0, n-1, 0, m-1$），按“右 $\rightarrow$ 下 $\rightarrow$ 左 $\rightarrow$ 上”四步循环，每走完一条边就收缩一格。

四条边的处理

- **左到右**：填 top 行，完后 top++
- **上到下**：填 right 列，完后 right–
- **右到左**：填 bottom 行，完后 bottom–
- **下到上**：填 left 列，完后 left++

关键点：后两步之前要先检查边界是否仍然合法，防止单行或单列时重复填充。

四条边走一圈，每走一条缩一格

螺旋矩阵：代码

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```
1  const int MAXN = 105;
2  int a[MAXN][MAXN];
3
4  void spiral(int n, int m) {
5      int top = 0, bottom = n - 1;
6      int left = 0, right = m - 1;
7      int num = 1;
8      while (top <= bottom && left <= right) {
9          // 左到右：填 top 行
10         for (int j = left; j <= right; j++) a[top][j] = num++;
11         top++;
12         // 上到下：填 right 列
13         for (int i = top; i <= bottom; i++) a[i][right] = num++;
14         right--;
15         // 右到左：填 bottom 行（先检查边界）
16         if (top <= bottom) {
17             for (int j = right; j >= left; j--) a[bottom][j] = num++;
18             bottom--;
```

螺旋矩阵：代码

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```
19          }
20          // 下到上：填 left 列（先检查边界）
21          if (left <= right) {
22              for (int i = bottom; i >= top; i--) a[i][left] = num++;
23              left++;
24          }
25      }
26  }
```

目录

1 二维数组基础

2 矩阵基本操作

3 螺旋矩阵

4 矩阵旋转

5 经典矩阵应用

6 数组模拟链表

目录

7 总结

例题：矩阵旋转 90 度

题目：给定一个 n 行 n 列的矩阵，将它顺时针旋转 90 度。要求原地旋转，不借助额外矩阵。

示例（$n = 3$），左为原矩阵，右为旋转后。

$$\begin{matrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{matrix} \quad \textbf{变成} \quad \begin{matrix} 7 & 4 & 1 \\ 8 & 5 & 2 \\ 9 & 6 & 3 \end{matrix}$$

矩阵旋转 90 度：解法

核心思路：先沿主对角线做转置，再把每一行左右翻转，两步原地完成。

关键步骤

- 第一步转置：对 i 小于 j 的位置交换 `a[i][j]` 与 `a[j][i]`
- 第二步翻转：每行 i，j 从 0 到 $n/2-1$，交换 `a[i][j]` 与 `a[i][n-1-j]`
- 全程原地操作，不需要额外矩阵

为什么正确：转置把 (i,j) 搬到 (j,i)；翻转再把 (j,i) 搬到 $(j,n-1-i)$，正好落在顺时针 90 度的位置。

先转置，再翻转，两步搞定

矩阵旋转 90 度：代码

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```cpp
const int MAXN = 105;
int a[MAXN][MAXN];

void rotate90(int n) {
    // 第一步：沿主对角线转置
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            swap(a[i][j], a[j][i]);
    // 第二步：每行左右翻转
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n / 2; j++)
            swap(a[i][j], a[i][n - 1 - j]);
}
```

转置 + 翻转；逆时针 90 度改成先转置再上下翻转

目录

1 二维数组基础

2 矩阵基本操作

3 螺旋矩阵

4 矩阵旋转

5 经典矩阵应用

6 数组模拟链表

目录

7 总结

例题：杨辉三角

题目：给定行数 n，输出杨辉三角的前 n 行。每行首尾为 1，其余数等于它左上方与右上方两数之和。

示例 $(n = 5)$

$$
\begin{array}{ccccc}
1 & & & & \\
1 & 1 & & & \\
1 & 2 & 1 & & \\
1 & 3 & 3 & 1 & \\
1 & 4 & 6 & 4 & 1
\end{array}
$$

杨辉三角：解法

核心思路：每个位置的值等于它上方两个位置的和，逐行从上往下推。

关键步骤

- 第 i 行第 0 列与第 i 列置 1（首尾固定为 1）
- 中间位置：`a[i][j] = a[i-1][j-1] + a[i-1][j]`
- 外层 i 从 0 到 $n-1$，内层 j 从 1 到 $i-1$
- 输出时每行只输出前 $i+1$ 个数

关键点：内层循环边界必须是 j 小于 i，不能到 n，否则会用到未初始化的位置。

上面两数相加，逐行往下推

杨辉三角：代码


```
const int MAXN = 105;
int a[MAXN][MAXN];

int main() {
    int n;
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        a[i][0] = 1;                    // 每行首位置为 1
        a[i][i] = 1;                    // 每行末位置为 1
        for (int j = 1; j < i; j++)
            a[i][j] = a[i-1][j-1] + a[i-1][j];
    }
    // 输出：每行只输出前 i+1 个数
    for (int i = 0; i < n; i++) {
        for (int j = 0; j <= i; j++)
            printf("%d ", a[i][j]);
        printf("\n");
    }
```

杨辉三角：代码


```
19      return 0;
20  }
```

例题：矩阵置零

题目：给定一个 n 行 m 列的矩阵，若某个元素为 0，则把该元素所在的整行和整列都置为 0。要求原地修改。

示例

$$\begin{matrix}1 & 1 & 1\\ 1 & 0 & 1\\ 1 & 1 & 1\end{matrix} \quad \text{变成} \quad \begin{matrix}1 & 0 & 1\\ 0 & 0 & 0\\ 1 & 0 & 1\end{matrix}$$

矩阵置零：解法

核心思路：分两遍走。第一遍记录所有 0 的位置，第二遍统一置零。

关键步骤

- 第一遍：遍历矩阵，把所有 0 的行号和列号存进两个数组
- 第二遍：遍历记录的坐标，把对应行和对应列全部置 0
- 两遍之间不改动原矩阵，避免边置零边丢失 0 的位置

为什么不能一遍：如果一边扫描一边把整行整列置零，刚被置零的格子会被当成本来的 0，错误地把整行整列都置零，结果覆盖过多。

先全部记下来，再统一开火

矩阵置零：代码


```cpp
const int MAXN = 105;
int a[MAXN][MAXN];
int ri[MAXN * MAXN], ci[MAXN * MAXN];

void setZeroes(int n, int m) {
    int cnt = 0;
    // 第一遍：记录所有 0 的位置
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            if (a[i][j] == 0) {
                ri[cnt] = i;
                ci[cnt] = j;
                cnt++;
            }
    // 第二遍：按记录统一置零
    for (int k = 0; k < cnt; k++) {
        for (int j = 0; j < m; j++) a[ri[k]][j] = 0;
        for (int i = 0; i < n; i++) a[i][ci[k]] = 0;
```

矩阵置零：代码


```
19        }
20    }
```

目录

1 二维数组基础

2 矩阵基本操作

3 螺旋矩阵

4 矩阵旋转

5 经典矩阵应用

6 数组模拟链表

目录

7 总结

数组模拟链表：是什么

问题：链表的节点要用指针串起来，学指针成本高，写链表还容易出错。

答案：用两个数组 `val` 和 `nxt` 模拟链表，完全不用指针。

两个数组的分工

- `val[i]`：第 i 个节点的值
- `nxt[i]`：第 i 个节点的下一个节点编号，-1 表示空
- `head`：头节点编号，-1 表示空链表

节点“指针”长什么样

- 就是一个整数，即数组下标
- 找下一个节点，用 `nxt[i]` 取出编号

数组下标代替指针，`nxt` 链起来就是链表

数组模拟链表：基本操作

遍历：从 head 出发，顺着 nxt 一路跳。

```
for (int p = head; p != -1; p = nxt[p])
```

头插法：新节点接到头部。

```
nxt[newn] = head; head = newn;
```

删除：把 pre 的 nxt 直接跳过 cur。

```
nxt[pre] = nxt[cur];
```

关键：所有操作都是下标寻址；新节点用 `++tot` 分配，从 0 号开始，依次往后用。

下标就是指针，nxt 就是 next

例题：合并两个有序链表

题目：给定两个按升序排列的链表 a 和 b，把它们合并成一个仍然升序的链表，返回合并后的头节点编号。

示例

- a 的值依次为 1, 3, 5；b 的值依次为 2, 4, 6
- 合并后：1, 2, 3, 4, 5, 6
- a 为空，b 为 1, 2；合并后：1, 2

合并两个有序链表：解法

核心思路：双指针各走一条链表，每次挑较小的节点接到结果链表尾部，直到一边为空。

关键步骤

- 用 p、q 分别指向 a 和 b 的头节点
- 用 head 记录结果头，tail 记录结果尾，初值都为 -1
- 每次比较 `val[p]` 和 `val[q]`，较小者作为 pick
- pick 接到 tail 之后，tail 移到 pick，对应指针前进一格
- 一方为空后，把另一方剩余部分整体接到 tail 后面

每次挑小的接龙，尾指针追踪末端

合并两个有序链表：代码


```
1  const int MAXN = 100005;
2  int val[MAXN], nxt[MAXN], tot;
3
4  int merge_list(int ha, int hb) {
5      int head = -1, tail = -1;
6      int p = ha, q = hb;
7      while (p != -1 && q != -1) {
8          int pick;
9          if (val[p] <= val[q]) { pick = p; p = nxt[p]; }
10         else { pick = q; q = nxt[q]; }
11         nxt[pick] = -1;                  // 摘下来接到结果
12         if (head == -1) head = pick; // 第一个节点成为新头
13         else nxt[tail] = pick;           // 尾节点接上新节点
14         tail = pick;
15     }
16     int rest = (p != -1) ? p : q;     // 剩下没走完的一段
17     if (rest != -1) {
18         if (head == -1) head = rest;
```

合并两个有序链表：代码

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```
19          else nxt[tail] = rest;
20      }
21      return head;
22  }
```

例题：反转链表

题目：给定一个链表的头节点，把它反转，返回新的头节点。要求原地修改，不新建节点。

示例

- 原链表：1, 2, 3, 4, 5
- 反转后：5, 4, 3, 2, 1
- 原链表为空或只有一个节点时，反转后不变

反转链表：解法

核心思路：三指针 pre、cur、next 一路推，每次把 cur 的 nxt 改成 pre，方向全部倒过来。

关键步骤

- **初始**：pre = -1，cur = head
- **循环条件**：cur 不等于 -1
- **循环体**：先记 next 等于 `nxt[cur]`，再令 `nxt[cur] = pre`
- **前进**：pre = cur，cur = next
- **循环结束**，pre 就是新链表头，返回 pre

关键点：必须先存 next 再改 `nxt[cur]`，否则链断掉就找不回下一个节点了。

先存后断，三指针一路推

反转链表：代码


```
1  int reverse_list(int head) {
2      int pre = -1;
3      int cur = head;
4      while (cur != -1) {
5          int next = nxt[cur];     // 先存下下一个节点
6          nxt[cur] = pre;          // 把当前节点的方向掰过来
7          pre = cur;               // pre 向前
8          cur = next;              // cur 向前
9      }
10     return pre;                  // pre 停在新头
11 }
```

pre 初始为 -1，正好作为新链表末尾的空指针

目录

1 二维数组基础

2 矩阵基本操作

3 螺旋矩阵

4 矩阵旋转

5 经典矩阵应用

6 数组模拟链表

目录

⑦ 总结

总结（一）：矩阵操作速查

问题	核心要点
矩阵转置	`b[j][i] = a[i][j]`，行列互换
螺旋矩阵	四条边按序走，走完缩一格，后两步查边界
矩阵旋转 90 度	先转置再左右翻转，原地完成
杨辉三角	`a[i][j] = a[i-1][j-1] + a[i-1][j]`
矩阵置零	两遍走，先记录所有 0，再统一置零

矩阵题核心就两件事：下标控制和边界判断

总结（二）：数组模拟链表

操作	关键写法
遍历	p 从 head 出发，按 nxt 跳，直到 -1
头插	新节点 nxt 指向 head，再更新 head
删除	令 pre 的 nxt 直接跳过 cur
合并有序	双指针选小，尾指针接龙
反转	三指针 pre、cur、next，先存后断

数组下标就是指针，nxt 就是 next，不需要学指针

总结（三）：常见易错点

矩阵类

- 行列下标搞反，转置写成 `a[i][j] = a[j][i]`（原地会覆盖数据）
- 螺旋矩阵后两步忘记检查边界，单行或单列时重复填
- 杨辉三角内层循环上界写成 n 而不是 i

链表类

- 反转链表忘记先存 next，断了链找不回
- 合并时忘记处理剩余部分，结果缺一截
- head 初始值 -1 时，第一次接节点要特殊处理

矩阵看边界，链表看指针顺序

附录：常见陷阱与调试技巧

矩阵

- 打印中间结果：用 `printf` 把每行打印出来，看行列是否对齐
- 小数据测试：$n = 1$ 或 $m = 1$ 时，螺旋、旋转是否仍正确

链表

- 遍历时把 head 当成固定头，不要改动它，用临时变量 p 走
- 删除后记得接到前驱上，否则链表被切断
- 空链表要单独处理，返回 -1

所有边界情况都用最小数据测一遍